NEROLITH

Decision Intelligence Platform

Advanced Flood Simulation

Algorithm Reference Document

Beyond the Baseline — Deep Computational Hydrology for Production-Grade Simulation

---

Version 2.0 · April 2026 · Internal Technical Reference · Confidential

C++ / Qt6 / Python / CUDA · FloodSim GIS · Nerolith Platform

Table of Contents

CATEGORY 01 — ADVANCED HYDRODYNAMIC SOLVERS

Godunov-Type Finite Volume Schemes

Nerolith currently uses Forward Euler explicit time integration on a structured grid. Godunov-type finite volume methods represent the state of the art in hyperbolic PDE solvers for the 2D Shallow Water Equations. They are the backbone of LISFLOOD-FP, GeoClaw, and Basilisk. These schemes are conservative by construction, handle discontinuities (shocks, wet-dry fronts) correctly, and are provably stable under the CFL condition.

1.1 The Finite Volume Framework

The 2D SWE in conservation form over a control volume V with boundary dV:

```
d/dt integral_V(U dV) + integral_dV(F(U) . n dS) = integral_V(S dV)
U = [h, hu, hv]^T (conserved variables)
F = [hu, hu^2 + gh^2/2, huv; (flux tensor)
hv, huv, hv^2 + gh^2/2]
S = [r-i, gh(S0x - Sfx), gh(S0y - Sfy)] (source terms)
```

1.2 Roe Approximate Riemann Solver

The Roe solver linearizes the Riemann problem at each cell interface using Roe-averaged variables. It is second-order accurate and handles transcritical flow.

```
Roe-averaged velocity: u_hat = (sqrt(hL)*uL + sqrt(hR)*uR) / (sqrt(hL) + sqrt(hR))
Roe-averaged celerity: c_hat = sqrt(g * (hL + hR) / 2)
Eigenvalues: lambda_1 = u_hat - c_hat, lambda_2 = u_hat, lambda_3 = u_hat + c_hat
Interface flux: F_Roe = 0.5*(F(UL) + F(UR)) - 0.5 * |A_hat| * (UR - UL)
where |A_hat| = R * |Lambda| * R^-1 (spectral decomposition)
```

1.3 HLL Solver (Harten-Lax-van Leer)

HLL is simpler than Roe and more robust for near-dry conditions:

```
Wave speed estimates: SL = min(uL - cL, uR - cR)
SR = max(uL + cL, uR + cR)
F_HLL = (SR*F(UL) - SL*F(UR) + SL*SR*(UR - UL)) / (SR - SL) if SL < 0 < SR
= F(UL) if SL >= 0
= F(UR) if SR <= 0
```

1.4 HLLC Solver (Contact Wave Restored)

HLLC restores the missing contact wave of HLL, critical for accurate sediment transport and tracer advection:

```
S* = (SL*hR*(uR-SR) - SR*hL*(uL-SL)) / (hR*(uR-SR) - hL*(uL-SL))
U*K = hK * ((SK - uK)/(SK - S*)) * [1, S*, vK]^T
F*K = F(UK) + SK*(U*K - UK)
F_HLLC = F*L if SL <= 0 <= S*
= F*R if S* <= 0 <= SR
```

	HLL Solver	HLLC Solver
	No	Yes
	Low	Medium

	Good	Good
	Fastest	Medium
	General flood	Tracer / sedimen
	Harten et al. 1983	Toro 1994 Shock

1.5 Improvement Over Nerolith Current

Current Nerolith uses diffusive wave with Forward Euler — no shock capturing, no wet-dry treatment. HLLC or HLL would: (1) correctly capture flood wave arrival times, (2) handle urban inundation wet-dry fronts without spurious oscillations, (3) conserve mass exactly. Estimated accuracy improvement: 35-60% in urban scenarios.

Complexity: High | C++ effort: 3-4 weeks | Reference: GeoClaw (github.com/clawpack/geoclaw)

CATEGORY 01 — ADVANCED HYDRODYNAMIC SOLVERS

MUSCL Schemes with Slope Limiters

First-order Godunov schemes are overly diffusive — flood fronts smear over multiple cells. MUSCL (Monotone Upstream-centered Scheme for Conservation Laws) achieves second-order spatial accuracy by reconstructing a linear profile within each cell before solving the Riemann problem at interfaces.

2.1 Linear Reconstruction

```
UL_(i+1/2) = U_i + 0.5 * phi(r_i) * (U_i - U_(i-1))
UR_(i+1/2) = U_(i+1) - 0.5 * phi(r_(i+1)) * (U_(i+2) - U_(i+1))
r_i = (U_i - U_(i-1)) / (U_(i+1) - U_i) (smoothness ratio)
```

2.2 Slope Limiters

Slope limiters prevent spurious oscillations (Gibbs phenomenon) near discontinuities:

```
MinMod: phi(r) = max(0, min(1, r))
Van Leer: phi(r) = (r + |r|) / (1 + |r|)
Superbee: phi(r) = max(0, min(2r,1), min(r,2))
MC: phi(r) = max(0, min((1+r)/2, 2, 2r))
```

Van Leer is recommended for flood simulation — it is smooth, compressive, and avoids the excessive diffusion of MinMod while being less aggressive than Superbee near shocks. Superbee is best when steep flood fronts must be preserved with high sharpness.

Complexity: Medium | C++ effort: 1-2 weeks on top of Godunov | Paper: Van Leer 1979 J. Comp. Phys.

CATEGORY 01 — ADVANCED HYDRODYNAMIC SOLVERS

GPU-Accelerated SWE Solvers

The SWE time-stepping loop is embarassingly parallel — each cell update depends only on its 4 neighbors from the previous timestep. This makes it ideal for GPU parallelization. GPU-accelerated flood solvers achieve 100-500x speedup over single-core CPU implementations, enabling real-time simulation of million-cell DEMs.

3.1 CUDA Thread Architecture

```
Grid layout: dim3 block(16, 16); // 256 threads per block
dim3 grid((cols+15)/16, (rows+15)/16);
__global__ void swe_kernel(float* h, float* hu, float* hv,
float* h_new, float* hu_new, float* hv_new,
int rows, int cols, float dt, float dx) {
int i = blockIdx.y * blockDim.y + threadIdx.y;
int j = blockIdx.x * blockDim.x + threadIdx.x;
// compute HLL flux at i+1/2 and i-1/2, j+1/2 and j-1/2
// update h_new[i][j], hu_new[i][j], hv_new[i][j]
}
```

3.2 Shared Memory Optimization

Loading halo cells into shared memory reduces global memory bandwidth by 4x for interior cells. Each 16x16 block loads an 18x18 tile (including 1-cell halos):

```
__shared__ float h_tile[18][18];
// Each thread loads its cell + boundary threads load halo cells
// Synchronize with __syncthreads() before flux computation
```

3.3 Performance Benchmarks (Published)

	Time per step	Speedup vs CPU
	0.8 ms	380x
	0.6 ms	210x
	2.1 ms	520x
	1.2 ms	160x

Reference: FullSWOF_GPU (Delestre et al. 2017) | Basilisk GPU branch | Complexity: High | Effort: 4-6 weeks

Adaptive Mesh Refinement (AMR)

AMR dynamically refines the simulation grid in regions of high water depth gradient (flood fronts, urban areas) and coarsens it in quiescent regions. This achieves the accuracy of a fine grid at a fraction of the computational cost.

4.1 Quad-Tree Refinement Criterion

```
Refinement indicator for cell (i,j) at level L:
eta(i,j) = max(|h(i+1,j) - h(i,j)|, |h(i,j+1) - h(i,j)|) / dx_L
Refine if: eta(i,j) > theta_refine (typically 0.1 m/m)
Coarsen if: eta(i,j) < theta_coarsen (typically 0.01 m/m)
Cell size at level L: dx_L = dx_0 / 2^L
Maximum levels: L_max = 4 gives 16x resolution locally
```

4.2 Conservative Prolongation and Restriction

When refining, parent cell conserved variables are distributed to children preserving total mass. When coarsening, children are averaged:

```
Prolongation (parent -> 4 children): h_child = h_parent (constant reconstruction)
Conservation check: sum(h_child * dx_child^2) = h_parent * dx_parent^2
Restriction (4 children -> parent): h_parent = sum(h_child * dx_child^2) / dx_parent^2
```

Reference: GeoClaw AMR (Berger & LeVeque 1998 SIAM) | *Basilisk quad-tree* | *Effort: 6-8 weeks*

CATEGORY 01 — ADVANCED HYDRODYNAMIC SOLVERS

LISFLOOD-FP Subgrid Channel Model

The LISFLOOD-FP subgrid model (Bates et al. 2010, Neal et al. 2012) achieves near-full SWE accuracy at 10-40x the speed by using a simplified inertial wave approximation on the 2D grid while resolving channels at subgrid scale. It is the most widely used flood model in research and operational forecasting worldwide.

5.1 Inertial Flow Equation

```
Simplified momentum (dropping convective acceleration):

q_(i+1/2)^(t+dt) = (q_(i+1/2)^t - g*h_flow * dt * (dh/dx)) / (1 + g*dt*n^2*|q^t| / h_flow^(7/3))

where:

q = unit discharge (m^2/s)

h_flow = max(z_i + h_i, z_(i+1) + h_(i+1)) - max(z_i, z_(i+1)) (flow depth)

n = Manning roughness coefficient

dh/dx = (h_(i+1) + z_(i+1) - h_i - z_i) / dx (water surface gradient)
```

5.2 Stability Condition

```
dt <= alpha * dx / sqrt(g * h_max)

Recommended: alpha = 0.7 (more stable than standard CFL = 0.5)

Typical timestep: 5-30 seconds for 30m DEM, 1-5 seconds for 5m DEM
```

The inertial approximation is valid for Froude numbers Fr < 0.5, which covers 99% of floodplain flow scenarios. For Himalayan torrents (Fr > 1), full SWE is needed.

Paper: Bates et al. 2010 J. Hydrology doi:10.1016/j.jhydrol.2010.03.027 | Effort: 2-3 weeks

CATEGORY 02 — ADVANCED TERRAIN AND FLOW ANALYSIS

Barnes Epsilon Filling and Breaching Algorithms

6.1 Priority-Flood with Epsilon (Barnes et al. 2014)

Wang and Liu (2006) fills pits to exactly the spill elevation, creating flat regions where flow direction is undefined. Barnes et al. (2014) adds a tiny epsilon increment at each step, ensuring every filled cell drains toward the depression outlet.

```
Standard Wang-Liu: Z'[p][q] = max(Z'[p][q], e)
Barnes Epsilon: Z'[p][q] = max(Z'[p][q], e + epsilon)
epsilon = EPSILON_SCALE * heap_insertion_count
EPSILON_SCALE ~ 1e-6 * (Z_max - Z_min) (relative to DEM range)
Result: No flat regions. Every cell has a unique downhill neighbor.
Flow accumulation works correctly on filled DEM with zero modifications.
```

6.2 Lindsay Breaching Algorithm (2016)

Instead of raising pit floors (filling), breaching cuts a channel through the lowest-cost path from a pit to the nearest drain point. Breaching is more hydrologically realistic for incised valleys — real drainage channels exist where breaching carves them.

```
Cost function for breaching path through cell (p,q):
cost(p,q) = w_elev * |Z[p][q] - Z_outlet| + w_dist * dist(p,q, outlet)
Least-cost path solved by Dijkstra from pit to drain.
All cells on path are lowered to create monotone drainage.
Hybrid strategy (recommended for Nerolith):
if breach_depth <= max_breach_depth -> breach
else -> fill (Wang-Liu epsilon)
```

Paper: Lindsay 2016 Hydrological Processes doi:10.1002/hyp.10648 | Effort: 1 week

D-Infinity Flow Direction — Tarboton 1997

D8 forces all flow from a cell into a single neighbor (1 of 8 directions). This creates parallel flow stripes on planar hillslopes that do not reflect reality. D-Infinity (Tarboton 1997) computes a continuous flow angle and partitions flow between two neighbors proportionally.

7.1 Triangular Facet Decomposition

```
Each cell has 8 triangular facets with its 8 neighbors.
For facet k formed by cell (i,j), neighbor 1, and neighbor 2:
s_max_k = steepest slope in facet k (computed from 3 elevations)
alpha_k = flow angle within facet (0 to pi/4 from cardinal direction)
Global flow angle: alpha = alpha_k* where k* = argmax(s_max_k)
Flow partition between two downslope neighbors:
p1 = (pi/4 - alpha_k*) / (pi/4) (fraction to cardinal neighbor)
p2 = alpha_k* / (pi/4) (fraction to diagonal neighbor)
p1 + p2 = 1 (mass conserved)
```

7.2 D-Infinity Flow Accumulation

```
A(i,j) = 1 + sum over all (p,q) that drain to (i,j): p_(p,q)->(i,j) * A(p,q)
where p_(p,q)->(i,j) is the flow proportion directed from (p,q) to (i,j)
Computed via topological sort in reverse flow order
```

D-Infinity produces significantly more realistic upslope area maps on hillslopes and flat terrain. For TWI computation, D-Infinity upslope area is strongly preferred over D8. On channelized terrain both methods converge.

Paper: Tarboton 1997 Water Resources Research doi:10.1029/96WR03137 | TauDEM implements this | Effort: 2 weeks

Compound Terrain Indices — SPI, STI, CTI

8.1 Stream Power Index (SPI)

```
SPI(i,j) = A(i,j) * tan(beta(i,j))
```

where:

A(i,j) = specific catchment area (upslope area / contour width, m^2/m)

beta = local slope angle (radians)

High SPI -> high erosive power of flowing water -> channel initiation zones

Threshold: SPI > 6 typically indicates channel initiation

8.2 Stream Transport Index (STI)

```
STI(i,j) = (A(i,j)/22.13)^0.6 * (sin(beta(i,j))/0.0896)^1.3
```

Predicts sediment transport capacity. High STI = high sediment flux.

Calibrated from USLE (Universal Soil Loss Equation) field data.

8.3 Compound Topographic Index (CTI)

```
CTI = TWI + SPI + STI (normalized composite)
```

Or weighted: CTI = w1*TWI + w2*SPI + w3*STI

Weights calibrated by regression against observed flood extent.

CTI is the recommended primary feature for flood susceptibility mapping

and for placing AI risk markers in the Qt renderer.

Reference: Moore et al. 1991 Hydrological Processes | Effort: 3 days (builds on existing TWI)

Physics-Informed Neural Networks (PINNs)

PINNs embed the governing PDEs (SWE continuity and momentum) directly into the neural network loss function. The network learns to satisfy both training data and the physical equations simultaneously, dramatically reducing data requirements and preventing physically impossible predictions.

9.1 PINN Architecture for SWE

```
Network input: (x, y, t) -> spatial coordinates and time
Network output: (h, u, v) -> predicted water depth and velocities
Total loss = L_data + lambda_pde * L_pde + lambda_bc * L_bc
L_data = MSE(h_pred, h_observed) at measurement points
L_pde = ||dh/dt + d(hu)/dx + d(hv)/dy||^2 (continuity)
+ ||d(hu)/dt + d(hu^2 + gh^2/2)/dx + ...||^2 (x-momentum)
+ ||d(hv)/dt + ... + d(hv^2 + gh^2/2)/dy||^2 (y-momentum)
Derivatives computed via automatic differentiation (autograd)
```

9.2 Training Strategy

PINNs for flood simulation use a two-phase training: (1) pre-train on synthetic SWE solutions (dam breaks, etc.) to initialize the physics residuals near zero, then (2) fine-tune on real DEM and observed inundation data. This avoids the optimization challenge of satisfying PDEs from random initialization.

```
Architecture: 6-8 hidden layers, 128-256 neurons, tanh activation
Optimizer: Adam with learning rate schedule (1e-3 -> 1e-5)
Collocation points: 50,000 random (x,y,t) samples in simulation domain
Training time: 4-12 hours on GPU for 10,000 epochs
```

Paper: Bihlo & Popovych 2022 J. Comp. Phys. | Rackauckas et al. 2020 | Effort: High — 8-12 weeks

CATEGORY 03 — ADVANCED ML AND AI FOR FLOOD

Graph Neural Networks for Watershed-Scale Prediction

The river network is naturally a directed graph — GNNs can learn complex upstream-downstream dependencies that traditional ML (random forest, MLP) cannot capture. GNNs have achieved state-of-the-art flood forecasting at continental scale (Google DeepMind's flood forecasting for India uses this approach).

10.1 Graph Construction from DEM

```
Nodes: one per watershed sub-catchment (region in Nerolith registry)
node features: [area, mean_elevation, mean_slope, TWI, SPI, soil_type]
Edges: directed from upstream to downstream (flow direction graph)
edge features: [flow_distance, elevation_drop, channel_width]
Dynamic features per timestep: [rainfall_t, rainfall_t-1, soil_moisture, antecedent_flow]
```

10.2 Message Passing

```
For each node v at layer l:
m_v^(l) = AGGREGATE({ MSG(h_u^(l-1), h_v^(l-1), e_uv) : u in N(v) })
h_v^(l) = UPDATE(h_v^(l-1), m_v^(l))
MSG and UPDATE are learnable MLPs
AGGREGATE = sum or attention-weighted sum
Output: h_v^(L) -> predicted peak flood depth for sub-catchment v
```

Paper: Moshe et al. 2020 NeurIPS | Google Flood Hub India | Effort: Medium-High — 4-6 weeks with PyTorch Geometric

CATEGORY 03 — ADVANCED ML AND AI FOR FLOOD

Surrogate Modelling — ML Emulation of SWE at 1000x Speed

A surrogate model (also called an emulator) is a fast ML model trained on the outputs of a slow physics solver. For Nerolith, this means: run the full SWE solver offline on thousands of (DEM, rainfall) pairs, then train a neural network to reproduce those outputs in milliseconds. At runtime, the surrogate replaces the SWE solver entirely.

11.1 Surrogate Architecture Options

	Output	Speed
	flood depth grid	1000x
	scalar metrics	10000x
	depth per cell	500x
	depth time series	200x
	node flood depth	5000x

11.2 U-Net Surrogate for Flood Grids

```
Input tensor: [DEM, flow_acc, TWI, rainfall] -> (4, H, W)
Output tensor: [flood_depth] -> (1, H, W)
U-Net encoder: 4 downsampling blocks (Conv + BatchNorm + ReLU)
U-Net decoder: 4 upsampling blocks with skip connections
Training loss: MSE(predicted_depth, SWE_depth) + TV_regularization
Training data: 10,000 SWE simulations with varied rainfall and DEMs
Training time: 6-12 hours on GPU
Inference: 3-8 ms per simulation (vs 5-30 min for full SWE)
```

Paper: Guo et al. 2021 Water Resources Research | Bentivoglio et al. 2022 | Effort: Medium — 3-4 weeks

CATEGORY 04 — ADVANCED RAINFALL AND HYDROLOGY MODELS

SCS Curve Number Runoff Method

The USDA SCS (now NRCS) Curve Number method is the industry standard for estimating surface runoff from rainfall in ungauged basins. It is simple, data-efficient, and calibrated against thousands of field measurements globally. It directly replaces the simplistic rainfall * flow_acc formula in Nerolith.

12.1 Runoff Equation

```
Actual runoff: Q = (P - Ia)^2 / (P - Ia + S) if P > Ia, else Q = 0
Initial abstraction: Ia = 0.2 * S (standard) or Ia = 0.05 * S (updated)
Potential retention: S = (1000 / CN) - 10 (inches) = 25400/CN - 254 (mm)
CN = Curve Number (0-100), depends on land use and soil group
```

12.2 Curve Number Table (Key Values)

	Soil B	Soil C
	98	98
	78	85
	61	74
	55	70
	100	100

Soil groups A (sandy, high infiltration) to D (clay, low infiltration). For South Asia: most Indo-Gangetic plain soils are group C or D.

Source: USDA NRCS National Engineering Handbook Part 630 | Effort: 3-4 days

CATEGORY 06 — URBAN FLOOD SPECIFIC

Dual-Drainage 1D-2D Coupled Models

Urban flood modelling requires coupling the underground sewer/stormwater network (1D) with surface overland flow (2D). Without this coupling, urban floods are significantly underestimated — sewer surcharge is a primary driver of pluvial flooding in cities.

13.1 1D Sewer Network (Saint-Venant 1D)

```
Continuity: dA/dt + dQ/dx = q_lat
Momentum: dQ/dt + d(Q^2/A)/dx + gA*dH/dx + gA*Sf = 0
where:
A = cross-sectional area of flow (m^2)
Q = discharge (m^3/s)
H = piezometric head (m)
Sf = friction slope = n^2*Q*|Q| / (A^2 * R^(4/3))
q_lat = lateral inflow from surface (m^2/s)
```

13.2 2D-1D Coupling Interface

```
Water enters sewer from surface when surface depth h_s > manhole rim elevation z_m:
q_in = Cd * A_manhole * sqrt(2g * (h_s + z_surface - z_m))
Water surcharges from sewer to surface when sewer head H > z_m:
q_out = Cd * A_manhole * sqrt(2g * (H - z_m))
Cd = discharge coefficient (~0.6 for circular manholes)
```

Reference: SWMM5 (EPA open source) | MIKE URBAN | Effort: High — 6-8 weeks for full 1D-2D coupling

CATEGORY 05 — REAL-TIME AND DATA ASSIMILATION

Ensemble Kalman Filter for Real-Time Flood State Estimation

The Ensemble Kalman Filter (EnKF) is the standard method for assimilating real-time observations (river gauges, satellite SAR, rain gauges) into a running simulation model. It enables the Nerolith simulation to correct itself using live data — the key capability for a digital twin.

14.1 EnKF Algorithm

```
Forecast step (run ensemble of N simulations with perturbed inputs):
x_i^f = M(x_i^a) + w_i, i = 1..N, w_i ~ N(0, Q)
Ensemble mean and covariance:
x_mean^f = (1/N) * sum(x_i^f)
P^f = (1/(N-1)) * sum((x_i^f - x_mean^f)(x_i^f - x_mean^f)^T)
Analysis step (update with observation y):
K = P^f * H^T * (H * P^f * H^T + R)^-1 (Kalman gain)
x_i^a = x_i^f + K * (y + epsilon_i - H * x_i^f)
where:
M = forward model (SWE solver)
H = observation operator (maps model state to observation space)
R = observation error covariance
epsilon_i ~ N(0, R) = observation perturbations
```

Practical ensemble size for Nerolith: N = 20-50 members. Each member is a separate SWE simulation with perturbed rainfall forcing. Observations: river gauge water level, satellite SAR binary flood extent, rain gauge rainfall.

Paper: Evensen 1994 J. Geophys. Res. | Matgen et al. 2010 for SAR assimilation | Effort: High — 6-8 weeks

CATEGORY 07 — ADVANCED RISK AND IMPACT ASSESSMENT

Fragility Curves and Depth-Damage Functions

Fragility curves relate flood depth (or velocity) to the probability of damage at or beyond a specified damage state. Depth-damage functions translate flood depth into economic loss. These are the bridge between physical simulation and actionable risk metrics for insurers, governments, and disaster managers.

15.1 Fragility Curve Formulation (Lognormal)

```
P(damage >= ds | depth = d) = Phi( (ln(d) - mu_ds) / sigma_ds )
where:
Phi = standard normal CDF
mu_ds = log-mean depth at damage state ds
sigma_ds = log-standard deviation
ds = damage states: slight / moderate / extensive / complete
Example (residential masonry, JRC 2017):
ds=slight: mu=0.25m, sigma=0.30
ds=moderate: mu=0.60m, sigma=0.35
ds=extensive: mu=1.20m, sigma=0.40
ds=complete: mu=2.50m, sigma=0.45
```

15.2 Depth-Damage Functions (Economic Loss)

```
Relative damage ratio (0 to 1):
alpha(d) = a * (1 - exp(-b*d)) (exponential form)
Economic loss for building i:
Loss_i = alpha(depth_i) * Value_i * Area_i
Total scenario loss:
Total_Loss = sum over all flooded buildings: Loss_i
JRC European damage functions (Huizinga et al. 2017):
Residential: alpha(d) = 1 - exp(-0.3*d)
Commercial: alpha(d) = 1 - exp(-0.4*d)
Industrial: alpha(d) = 1 - exp(-0.25*d)
```

Source: JRC Technical Report EUR 28386 (Huizinga et al. 2017) | HAZUS-MH MR4 | Effort: 1 week

SECTION 23 — IMPLEMENTATION ROADMAP

Prioritized Algorithm Implementation Plan for Nerolith

The following roadmap prioritizes algorithms by the ratio of accuracy improvement to engineering effort. P1 items have the highest impact and lowest cost. Each phase builds on the previous.

Priority	Algorithm	Category	Effort	Impact	Phase
P1 — Critical	Barnes Epsilon Filling	Terrain	3 days	High	Now
P1 — Critical	Lindsay Breaching	Terrain	1 week	High	Now
P1 — Critical	SCS Curve Number	Hydrology	4 days	Very High	Now
P1 — Critical	SPI + STI Indices	Terrain	3 days	High	Now
P2 — High	LISFLOOD-FP Inertial	Solver	3 weeks	Very High	Month 1-2
P2 — High	D-Infinity Flow Dir	Terrain	2 weeks	High	Month 1-2
P2 — High	HLL Godunov Solver	Solver	4 weeks	Very High	Month 2-3
P2 — High	MUSCL Van Leer Limiter	Solver	2 weeks	High	Month 3
P3 — Medium	GNN Watershed Model	ML/AI	6 weeks	Very High	Month 3-5
P3 — Medium	U-Net Surrogate Model	ML/AI	4 weeks	High	Month 4-5
P3 — Medium	Dual-Drainage 1D-2D	Urban	8 weeks	Very High	Month 5-7
P3 — Medium	AMR Quad-Tree	Solver	8 weeks	High	Month 6-8
P4 — Research	GPU CUDA SWE	Solver	6 weeks	Extreme	Month 8-10
P4 — Research	EnKF Data Assimilation	Real-time	8 weeks	Very High	Month 9-12
P4 — Research	PINNs for SWE	ML/AI	12 weeks	Very High	Month 10-14

SECTION 24 — KEY ACADEMIC PAPERS 2015-2025

Essential Reading for Nerolith Algorithm Development

Bates et al. 2010 | J. Hydrology
A simple inertial formulation of the SWE for efficient 2D flood inundation modelling — the LISFLOOD-FP subgrid foundation.

Neal et al. 2012 | Water Resour. Res.
How simple is too simple? Optimal complexity for urban flood inundation modelling.

Barnes et al. 2014 | Computers & Geosciences
Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models.

Lindsay 2016 | Hydrol. Processes
Efficient hybrid breaching-filling sink removal methods for flow path enforcement in digital elevation models.

Tarboton 1997 | Water Resour. Res.
A new method for the determination of flow directions and upslope areas in grid digital elevation models.

Huizinga et al. 2017 | JRC Tech. Report
Global flood depth-damage functions: Methodology and the database with guidelines.

Moshe et al. 2020 | NeurIPS
HydroNets: Leveraging river structure for hydrological modelling using graph neural networks.

Bentivoglio et al. 2022 | Water Resour. Res.
Deep learning methods for flood mapping — a review of existing applications and future research directions.

Bihlo & Popovych 2022 | J. Comp. Phys.
Physics-informed neural networks for the shallow water equations on the sphere.

Guo et al. 2021 | Water Resour. Res.
Data-driven flood emulation: Speeding up urban flood predictions by deep learning.

Toro 2009 | Springer
Riemann Solvers and Numerical Methods for Fluid Dynamics — the definitive Godunov/HLLC reference.

Matgen et al. 2010 | Adv. Water Resour.
Towards the sequential assimilation of SAR-derived water stages into hydraulic models using the particle filter.

Evensen 2009 | Springer
Data Assimilation: The Ensemble Kalman Filter — the standard reference for EnKF theory and practice.

Berger & LeVeque 1998 | SIAM J. Numer. Anal.
Adaptive mesh refinement using wave-propagation algorithms for hyperbolic systems.

Quinn et al. 1991 | Hydrol. Processes
The prediction of hillslope flow paths for distributed hydrological modelling using digital terrain models.

APPENDIX — OPEN SOURCE FLOOD SIMULATION CODEBASES

Reference Implementations for Nerolith

LISFLOOD-FP | C++ | github.com/uob-hydrology/lisflood-fp
Algorithms: Inertial SWE, subgrid, GPU, SAR assimilation

The most widely cited 2D flood model. Study the inertial solver and GPU kernel for Nerolith Phase 2.

Basilisk | C (quad-tree) | basilisk.fr
Algorithms: Godunov SWE, AMR, MPI parallel, VOF

Stéphane Popinet's solver. Best reference for AMR implementation and Godunov schemes on adaptive grids.

GeoClaw | Fortran/Python | github.com/clawpack/geoclaw
Algorithms: Godunov AMR, Roe solver, storm surge

LeVeque group. Best reference for Roe/HLL solvers and multi-level AMR with conservation.

SWMM5 | C | github.com/USEPA/Stormwater-Management-Model
Algorithms: 1D sewer network, Saint-Venant 1D

EPA stormwater model. Essential reference for 1D sewer network coupling (dual-drainage).

TauDEM | C++/MPI | github.com/dtarb/TauDEM
Algorithms: D-infinity, parallel flow acc, watershed

Tarboton's own implementation of D-infinity and parallel flow accumulation. Direct reference code.

WhiteboxTools | Rust | github.com/jblindsay/whitebox-tools
Algorithms: Breaching, filling, TWI, curvature, GIS

Lindsay's own tool — contains his breaching algorithm. Best reference for all terrain analysis.